



Práctica 2

Aceleración en GPU (OpenACC)

Instrucciones para la entrega:

Para la evaluación de esta práctica se debe entregar el código desarrollado y una memoria en formato PDF contestando a las preguntas del enunciado y mostrando los bloques de código relevantes, acompañados de una explicación de dicho código que describa como resuelve el problema planteado.

Para la entrega de dicha práctica se deberán subir los archivos correspondientes a las diferentes entregas creadas para ello: 1 entrega para la memoria, otra entrega para el proyecto de código, comprimido en formato ZIP.

Contexto:

OpenACC es un estándar de programación paralela desarrollado por Cray, CAPS, Nvidia y PGI. Su objetivo es simplificar la tarea de programación a desarrolladores de C, C++ y Fortran para emplear aceleración por hardware en sus cómputos.

Se basa en directivas de compilación, y su desarrollo se inspira en OpenMP para ofrecer a los desarrolladores una sintaxis similar.

Se puede encontrar más información en los documentos presentados en clase y en los siguientes enlaces:

- [Especificación de estándar OpenACC](#)
- <https://openacc-best-practices-guide.readthedocs.io/>

Compilador Nvidia HPC

De acuerdo con poder utilizar openACC para paralelizar nuestros procesos en GPU, debemos instalar el compilador adecuado. Para esta práctica instalaremos el compilador de Nvidia para computación de alto rendimiento, última versión publicada.

Una vez instalado el compilador, el siguiente paso para poder usar OpenACC es importar la cabecera de la librería en nuestro código.

Dentro de nuestro proyecto, hay que descomentar la línea relativa al proyecto de computación en streaming y que está incluida en el CMakeList de la raíz del proyecto:



```
# Compile executable for shared memory
add_subdirectory( computeShared )
add_subdirectory( computeDist )
add_subdirectory( computeStream )
```

Dentro de ese subdirectorio hay que definir correctamente el archivo `CMakeLists.txt`, para definir el compilador y las librerías correspondientes.

Requisitos de la práctica

1. Configuración del entorno (2 puntos):

- Configuración del CMake para que compile usando GPU o CPU usando un flag
- Crear un ejecutable en CPU y otro en GPU para la Práctica 2.
- Conectarse a la máquina en remoto y comprobar la tarjeta gráfica de la que dispone el equipo. Describir en la memoria la capacidad de cómputo de dicha tarjeta gráfica.

2. Ejercicio de tratamiento de datos 1 (3 puntos):

- Mediante computación asistida en aceleradores, realiza un recuento del número de avistamientos de cada Pokémon recogidos en el conjunto de datos de la práctica 1. Gestiona de forma manual los pasos de memoria a GPU y su vuelta. Justifica los puntos en los que has realizado la paralelización. Comprueba que el total calculado coincide con el número total de avistamientos. Realizar mediciones de rendimiento y comparar con los resultados de la práctica 1 (Paralelismo con OpenMP y con OpenMPI).

3. Ejercicio de tratamiento de datos 2 (5 puntos):

- El algoritmo **k-means** es un método de agrupamiento no supervisado que divide un conjunto de datos en k grupos (clústers) según la similitud entre sus puntos. Funciona iterativamente: primero elige k centroides iniciales, asigna cada punto al centro más cercano y luego recalcula los centroides como la media de los puntos en cada grupo. Este proceso se repite hasta que las asignaciones no cambian significativamente o se alcanza un criterio de convergencia, buscando minimizar la suma de distancias dentro de cada clúster.
Implementa dicho algoritmo paralelizado en GPU usando openACC con gestión manual de la memoria.



Posteriormente, aplica dicha implementación sobre los datos de posición del conjunto de datos de la práctica 1, generando 5 clústers como opciones de continente tiene el conjunto original. Agrega los nuevos valores de etiquetas generados como una columna nueva al conjunto de datos.